# Ejercicio 6: Polimorfismo vía Interfaces

Análisis y diseño · Temario C

Sistema de integración con APIs externas para una plataforma de comercio electrónico



| | |
|---|---|
| Nombre | Fabio Calvo |
| Carné | 262256 |
| Curso / sección | CC2008 – Programación Orientada a Objetos |
| Repositorio | https://github.com/astxrfr/Ejercicio6ECommerce.git |



## 1. Análisis del problema

### 1.1 Descripción

El backend de un e-commerce debe comunicarse con servicios externos de naturaleza distinta: pasarelas de pago (tarjeta, transferencia, billetera digital), proveedores de logística y certificadores de Factura Electrónica en Línea (FEL). Cada servicio ofrece capacidades diferentes: unos cobran y reintegran, otros generan documentos legales, otros solo cotizan o rastrean. El reto es que el sistema central los trate de forma uniforme, sin conocer su tipo concreto, y que se puedan agregar servicios nuevos sin modificar el motor principal.

La solución propuesta modela lo que cada servicio **es** mediante una jerarquía de herencia, y lo que cada servicio **puede hacer** mediante interfaces (contratos). Todos los objetos viven en una única lista polimórfica.

### 1.2 Requisitos funcionales

| ID | Requisito | Cómo se cubre |
|---|---|---|
| RF1 | Carga inicial con al menos 10 integraciones de naturaleza diversa. | `Main.cargaInicial` registra 11 (pago, logística y FEL). |
| RF2 | Listar las integraciones activas con su estado de conexión. | Opción 1 del menú: `ControladorIntegraciones.listarActivas()`. |
| RF3 | Buscar una API por identificador y mostrar sus parámetros de configuración. | Opción 2: `buscarPorId()` + `getParametrosConfiguracion()`. |
| RF4 | Ordenar el catálogo por latencia promedio (ms). | Opción 3: `ordenarPorLatencia()` usando `Comparable`. |
| RF5 | Operar cualquier integración mediante los contratos Transaccional, Documentable y Auditable. | Opción 4 (simulación de checkout) y métodos `procesarCobro/Reintegro`, `generarDocumento`, `consultarAuditoria`. |
| RF6 | Si una API no soporta una operación (p. ej. reintegro en logística), ignorarla sin interrumpir la compra ni lanzar excepciones. | Verificación por interfaz (`instanceof`) antes de operar; ver sección 3.5. |
| RF7 | Integrar nuevos servicios sin modificar la lógica central. | Basta con instanciar la nueva clase y agregarla al catálogo; ver sección 3.10. |

### 1.3 Restricciones del enunciado

- Una **sola** lista polimórfica que guarde todas las integraciones.
- Contratos estandarizados mediante interfaces (Transaccional, Documentable, Auditable); aprovechar herencia y polimorfismo.
- Patrón MVC y separación de responsabilidades; principios de diseño vistos en clase.

- Overloading donde sea necesario, con interfaces como tipo de parámetro.
- Interfaz `Comparable` para ordenar la lista polimórfica; `toString` y `equals` sobrescritos; encapsulación con visibilidad y getters/setters.
- Diseñar primero; si la implementación exige cambios, rediseñar y documentarlos con comentarios.

### 1.4 Supuestos adoptados

- Una integración está **activa** si su estado es distinto de DESCONECTADA. El estado puede ser CONECTADA, DEGRADADA o DESCONECTADA.
- El identificador es único y no distingue mayúsculas de minúsculas (se normaliza a mayúsculas). El catálogo rechaza duplicados.
- La latencia promedio se mide en milisegundos y no puede ser negativa.
- Las operaciones son simuladas en memoria: no hay llamadas reales de red. Cada operación deja un evento en el historial de auditoría cuando la API es Auditable.
- Se agrega una opción 4 al menú (simular checkout) para demostrar el polimorfismo; no forma parte de los tres requisitos de menú del enunciado.

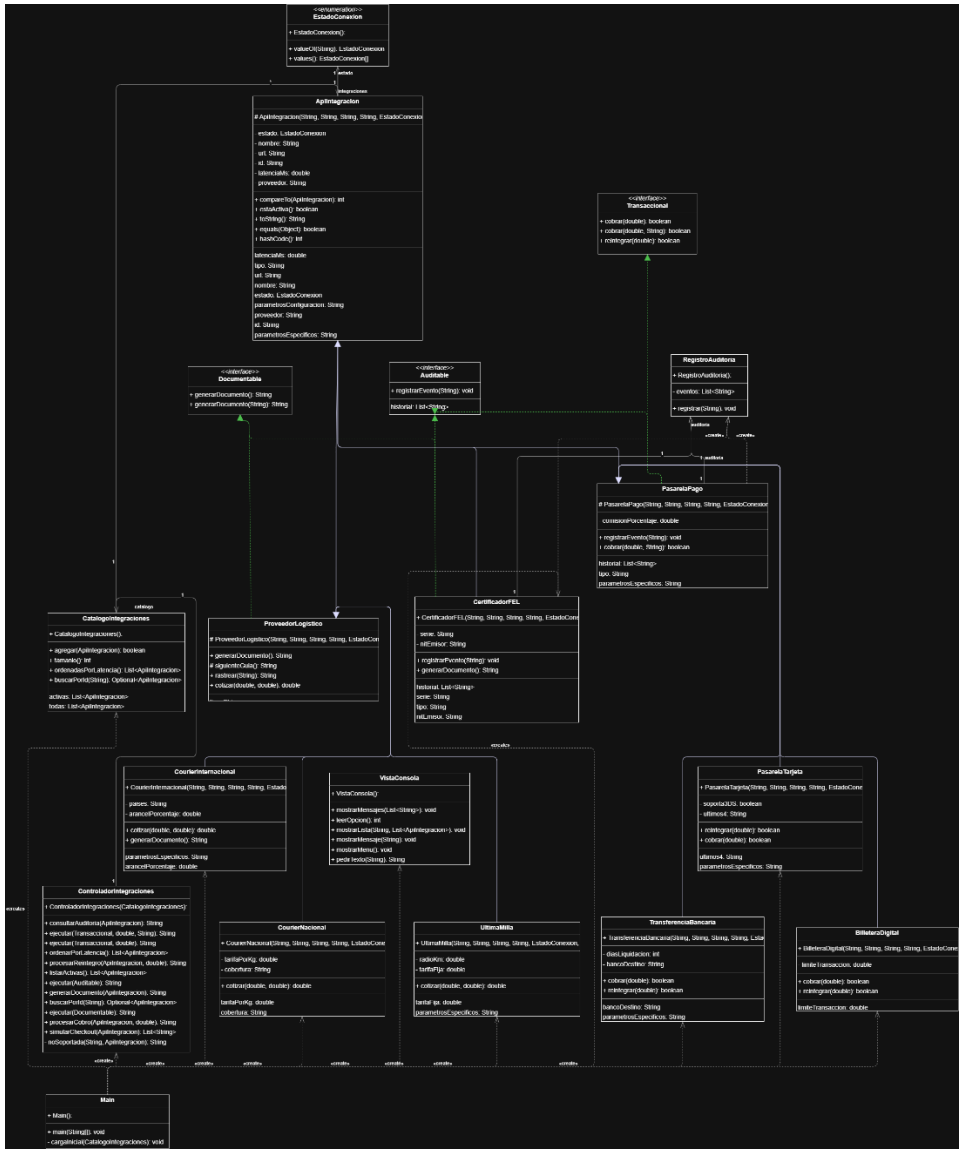
## 2. Identificación de abstracciones

Se analizaron los sustantivos y verbos del enunciado para decidir qué se modela como clase, como interfaz y como comportamiento.

| Concepto del enunciado | Elemento del diseño | Justificación |
|---|---|---|
| Servicio externo / API | `ApiIntegracion` (clase abstracta) | Comparte identidad, estado de conexión y latencia; es el tipo de la lista polimórfica. |
| Cobro y reintegro | `Transaccional` (interfaz) | Capacidad financiera que solo tienen las pasarelas. |
| Documentos legales (XML) | `Documentable` (interfaz) | Capacidad compartida por logística (guías) y FEL (facturas), que no tienen un padre común más específico. |
| Rastro de operaciones | `Auditable` (interfaz) | Capacidad compartida por pasarelas y FEL. |
| Tarjeta, transferencia, billetera | `PasarelaTarjeta`, `TransferenciaBancaria`, `BilleteraDigital` | Cada una implementa cobro y reintegro con reglas propias. |
| Envíos, rastreo, cotización | `ProveedorLogistico` y sus tres hijas | Cotizan, rastrean y generan guías; no son transaccionales. |
| Certificador FEL | `CertificadorFEL` | Genera facturas y deja evidencia. |
| Registro de integraciones | `CatalogoIntegraciones` | Encapsula la única lista polimórfica. |
| Ordenar por latencia | `Comparable<ApiIntegracion>` | Orden natural de la jerarquía. |
| Administrador por consola | `VistaConsola`, `ControladorIntegraciones` | Separación Vista / Controlador. |

# 3. Diseño

## 3.1 Diagrama de clases del modelo (UML)



## 3.2 Interfaces (contratos)

| Interfaz | Métodos | Implementada por |
|---|---|---|
| `Transaccional` | `cobrar(double)`, `cobrar(double, String)`, `reintegrar(double)` | `PasarelaPago` y sus hojas |
| `Documentable` | `generarDocumento()`, `generarDocumento(String formato)` (método por defecto: XML o TEXTO) | `ProveedorLogistico`, `CertificadorFEL` |
| `Auditable` | `registrarEvento(String)`, `getHistorial()` | `PasarelaPago`, `CertificadorFEL` |
| `Comparable<ApiIntegracion>` | `compareTo(ApiIntegracion)` | `ApiIntegracion` |

## 3.3 Clases y su comportamiento distintivo

| Clase | Tipo / hereda de | Atributos propios | Comportamiento propio |
|---|---|---|---|
| `ApiIntegracion` | Abstracta | id, nombre, proveedor, url, estado, latenciaMs | Método plantilla `getParametrosConfiguracion()`; `compareTo`, `equals`, `hashCode`, `toString`. |
| `PasarelaPago` | Abstracta / ApiIntegracion | comisionPorcentaje, auditoria | Implementa Auditable por composición; `cobrar(monto, moneda)` registra la moneda y delega. |
| `PasarelaTarjeta` | PasarelaPago | ultimos4, soporta3DS | El cobro incluye la comisión; el reintegro regresa a la tarjeta. |
| `TransferenciaBancaria` | PasarelaPago | bancoDestino, diasLiquidacion | El cobro queda pendiente hasta la liquidación; el reintegro se programa. |
| `BilleteraDigital` | PasarelaPago | limiteTransaccion | Rechaza cobros que superan el límite; reintegro inmediato. |
| `ProveedorLogistico` | Abstracta / ApiIntegracion | guiasEmitidas | `cotizar` abstracto, `rastrear`, guía de envío en XML. |
| `CourierNacional` | ProveedorLogistico | tarifaPorKg, cobertura | Cotiza por peso. |
| `CourierInternacional` | ProveedorLogistico | tarifaPorKg, arancelPorcentaje, paises | Cotiza sumando aranceles; su guía incluye sección de aduana. |
| `UltimaMilla` | ProveedorLogistico | radioKm, tarifaFija | Tarifa fija dentro del radio; devuelve -1 fuera de cobertura. |
| `CertificadorFEL` | ApiIntegracion | nitEmisor, serie, facturasEmitidas, auditoria | Genera la factura XML y registra cada certificación. |
| `RegistroAuditoria` | Clase auxiliar | eventos | Historial reutilizado por composición (evita duplicar código). |
| `CatalogoIntegraciones` | Modelo | integraciones: List<ApiIntegracion> | Agregar (sin duplicados), filtrar activas, buscar por id, ordenar. |

### 3.4 Única estructura polimórfica

Los once objetos se almacenan en un solo atributo privado, `List<ApiIntegracion>`, dentro de `CatalogoIntegraciones`. Las vistas de la lista (solo activas, ordenadas por latencia) son copias temporales que se construyen al consultar; no son estructuras de almacenamiento y el sistema no mantiene ninguna otra colección de integraciones. La lista expuesta hacia afuera es de solo lectura (`Collections.unmodifiableList`), de modo que solo el catálogo puede agregar elementos.

### 3.5 Polimorfismo vía interfaces

Se usan dos formas de polimorfismo:

- **Despacho dinámico de métodos sobrescritos.** Al invocar `cobrar()` a través de `Transaccional`, o `getTipo()`, `getParametrosEspecificos()` y `generarDocumento()` a través de la referencia `ApiIntegracion`, se ejecuta la versión de la clase real del objeto.
- **Despacho por capacidades.** El controlador pregunta qué contrato cumple el objeto y opera solo si lo cumple.

Como `ProveedorLogistico` no implementa Transaccional, el reintegro simplemente no existe para él: no hay un método vacío ni un `UnsupportedOperationException`. El proceso de compra continúa y el administrador recibe un mensaje informativo. El mismo patrón se aplica a documentos (solo se genera si la API es Documentable) y a auditoría (solo sobre Auditable).

### 3.6 Overloading

| Ubicación | Firmas sobrecargadas | Propósito |
|---|---|---|
| `Transaccional` / `PasarelaPago` | `cobrar(double)` y `cobrar(double, String)` | Cobro en la moneda por defecto o en una indicada. |
| `Documentable` | `generarDocumento()` y `generarDocumento(String)` | Documento XML o en texto plano. |
| `ControladorIntegraciones` | `ejecutar(Transaccional, double)`, `ejecutar(Transaccional, double, String)`, `ejecutar(Documentable)`, `ejecutar(Auditable)` | Comportamiento según el contrato recibido como parámetro. |

### 3.7 Implementación de Comparable

`ApiIntegracion` implementa `Comparable<ApiIntegracion>`. El orden natural es por latencia ascendente y, ante empate, por id. Así `Collections.sort(lista)` ordena toda la jerarquía con una sola definición, y el orden es consistente con `equals` (dos objetos son iguales solo si comparten id).

### 3.8 Patrón MVC y separación de responsabilidades

Figura 2. Arquitectura MVC. Línea punteada con flecha abierta: dependencia. Rombo hueco: agregación.

| Capa | Clases | Responsabilidad | Lo que NO hace |
|---|---|---|---|
| Modelo | `ApiIntegracion` y subclases, interfaces, `CatalogoIntegraciones` | Datos, reglas de cada integración, almacenamiento y orden de la lista. | No imprime ni lee de consola. |
| Vista | `VistaConsola` | Menú, lectura de opciones y texto, presentación de listas y mensajes. | No contiene reglas de negocio. |
| Controlador | `ControladorIntegraciones` | Traduce las acciones del menú a operaciones del modelo y decide si una operación aplica según el contrato. | No conoce clases concretas ni imprime. |
| Driver | `Main` | Carga inicial, ensamblado de las tres capas y ciclo del menú. | No contiene lógica de checkout. |

### 3.9 Principios de diseño aplicados

| Principio | Aplicación en el diseño |
|---|---|
| Responsabilidad única | Cada clase tiene un motivo de cambio: la vista solo presenta, el controlador solo coordina, el catálogo solo guarda y ordena, RegistroAuditoria solo registra eventos. |
| Abierto / cerrado | Se extiende creando nuevas subclases; el controlador, el catálogo y la vista no se modifican. |
| **Principio** | **Aplicación en el diseño** |
| Sustitución de Liskov | Cualquier subclase puede usarse donde se espere `ApiIntegracion` sin romper el comportamiento. |
| Segregación de interfaces | Tres contratos pequeños en lugar de uno grande; ninguna clase implementa métodos que no puede cumplir. |
| Inversión de dependencias | El controlador depende de abstracciones (`ApiIntegracion` y las interfaces), no de clases concretas. |
| Composición sobre herencia | El historial de auditoría se comparte con `RegistroAuditoria` en lugar de heredarlo. |
| Método plantilla | `getParametrosConfiguracion()` arma los datos comunes y delega los específicos en `getParametrosEspecificos()`. |
| Encapsulación | Atributos privados, getters/setters solo donde corresponde (el id no tiene setter), validaciones en constructor y setters. |

### 3.10 Extensibilidad

Cuando la empresa contrate la startup de entregas de última milla basta con una línea en la carga inicial; ya es un `ProveedorLogistico` y por tanto cumple los mismos contratos:

```
catalogo.agregar(new UltimaMilla("LOG-003", "RápidoYa", "RápidoYa",
"https://api.rapidoya.example", EstadoConexion.CONECTADA, 75, 15, 25));
```

Si la nueva integración fuera de otro tipo (por ejemplo, un servicio de cobros distinto), se crea una subclase que implemente los contratos que cumple, sin cambiar el controlador. Las tres opciones del menú y el checkout funcionan de inmediato.

### 3.11 Buenas prácticas de programación

- `toString()` sobrescrito en `ApiIntegracion` (formato tabular que usan los listados).
- `equals()` y `hashCode()` por identificador.
- Atributos `private`, métodos auxiliares `protected`, getters y setters con validación.
- Comentarios de clase y de métodos clave; encabezado en el driver program.

## 4. Carga inicial del catálogo

| ID | Nombre | Clase | Estado | Latencia (ms) |
|---|---|---|---|---|
| TARJ-001 | PagoSeguro GT | PasarelaTarjeta | CONECTADA | 180 |
| TARJ-002 | CardLatam | PasarelaTarjeta | DEGRADADA | 240 |
| TRF-001 | BancoNorte ACH | TransferenciaBancaria | CONECTADA | 420 |
| TRF-002 | InterBanco | TransferenciaBancaria | CONECTADA | 390 |

| | | | | |
|---|---|---|---|---|
| BIL-001 | PayWallet | BilleteraDigital | CONECTADA | 95 |
| BIL-002 | QuetzalPay | BilleteraDigital | CONECTADA | 130 |
| LOG-001 | ExpressGT | CourierNacional | CONECTADA | 310 |
| LOG-002 | GlobalCargo | CourierInternacional | DEGRADADA | 680 |
| LOG-003 | RápidoYa | UltimaMilla | CONECTADA | 75 |
| FEL-001 | CertiFEL | CertificadorFEL | CONECTADA | 260 |
| FEL-002 | FacturaYa | CertificadorFEL | DESCONECTADA | 0 |

Son 11 integraciones registradas, de las cuales 10 están activas. La desconectada permite comprobar que la opción 1 filtra por estado mientras que la opción 3 ordena el catálogo completo.

## 5. Flujo del menú y casos de prueba

| Opción | Flujo | Resultado esperado |
|---|---|---|
| 1 | Vista → Controlador.listarActivas() → Catálogo.getActivas() → Vista.mostrarLista() | Tabla con las 10 integraciones activas y su estado. |
| 2 | Vista pide el ID → Controlador.buscarPorId() → getParametrosConfiguracion() | Parámetros comunes y específicos de la API; mensaje claro si el ID no existe. |
| 3 | Controlador.ordenarPorLatencia() → Collections.sort (Comparable) | 11 integraciones de menor a mayor latencia. |
| 4 | Controlador.simularCheckout(api): cobro, reintegro, documento, auditoría | Cada operación se ejecuta o se ignora según los contratos de la API. |

| Caso de prueba | Entrada | Resultado verificado |
|---|---|---|
| Logística ignora operaciones financieras | Opción 4 con LOG-003 | Cobro, reintegro y auditoría: «no soportada… se ignora». El documento (guía) sí se genera. Sin excepciones. |
| FEL genera documento y auditoría | Opción 4 con FEL-001 | Cobro y reintegro ignorados; se genera la factura XML y el historial muestra la certificación. |
| Pasarela completa | Opción 4 con BIL-001 | Cobro y reintegro aprobados; documento ignorado; el historial registra ambas operaciones. |
| Orden por latencia | Opción 3 | Orden ascendente de latencia con las 11 integraciones. |
| ID inexistente | Opción 2 con «xxx» | Mensaje «No existe una integración con ID…» sin romper el programa. |
| Opción inválida | Opción 9 | Mensaje de opción inválida y el menú se vuelve a mostrar. |